18. Hashing and Collisions

T.V. GEETHA

epgp books

Welcome to the e-PG Pathshala Lecture Series on Data Structures. We have understood the basic concept Hashing and Hashing functions. In this module we will discussone very important concept associated with hashing – collisions and techniques for collision resolution.

Learning Objectives

The learning objectives of the module are as follows:

- To explain the concept of collision
- To discuss the different collision resolution methods
- To describe the direct addressing, chaining and open addressing methods

18.1 Introduction

Before we discuss collision resolution let us understand a simple method of

hashing called direct addressing where we assume that the key values are distinct and each key is drawn from a universe U = {0, 1, . . . , m − 1} (Figure 18.1). The idea is to store the items in an array, indexed by keys. The Direct-address table representation uses an array T[0 . . . m − 1] where each **slot**, or bucket, in T corresponds to a key in U. For an element x with key k, a pointer to x (or x itself) will be placed in location T[k]. If there are no elements with the key k in the set, that is T[k] is empty, it is represented by NIL. A fixed size array storing the items can be considered as an ideal hash table. Collisions occur when an element is inserted, it hashes to the same value as an already inserted element. For a given set K of keys if the number of keys is lesser or equal to the number of mapped values, the occurrence of collisions depends on the hash function but however when the number of keys is greater than the number of mapped values, the possibility of collisions is certain. Avoiding collisions completely is hard, however good the hash function. Figure 18.2 shows an example of collision where the element 4567 hashes to the table bucket 22 but that bucket is already occupied by another element 7597.

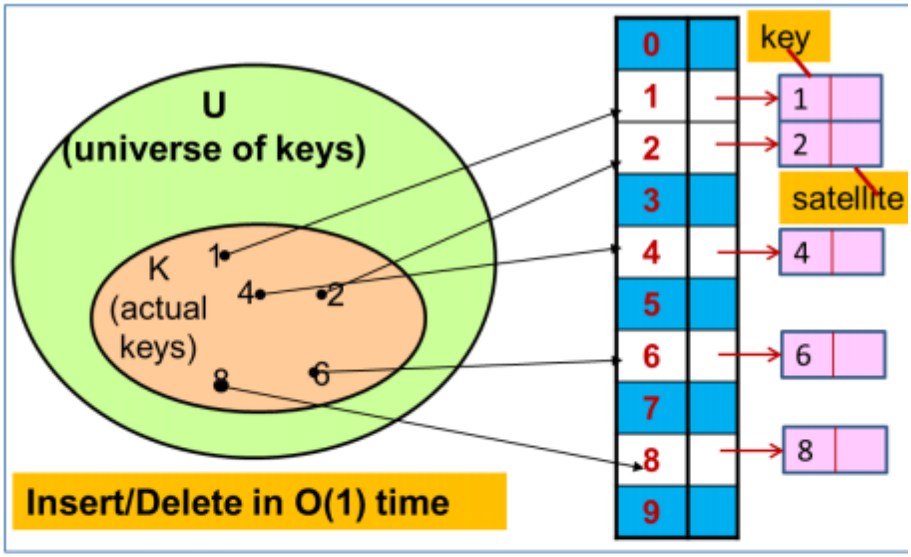

Figure 18.1 Direct Addressing

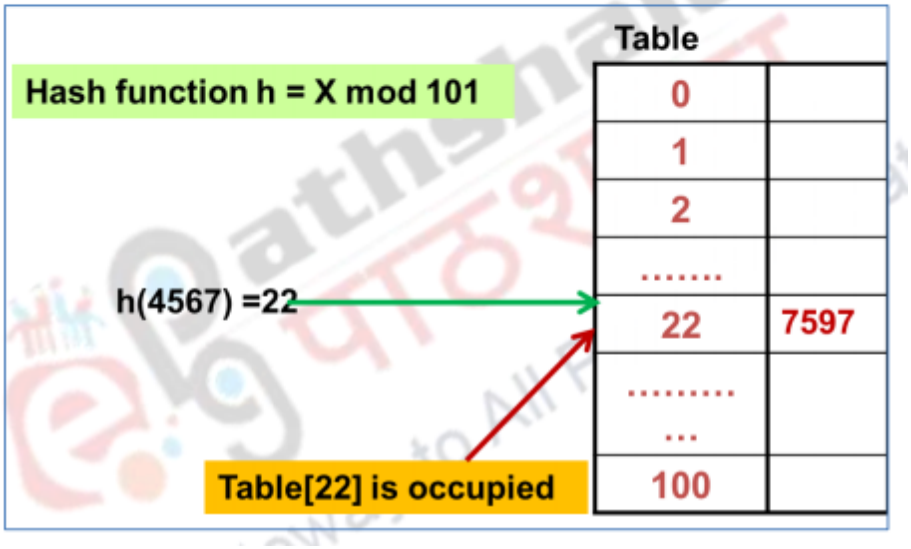

Figure 18.2An Example of Collision

18.2 Collision Resolution

Collisions that occur during hashing need to be resolved. In order to tackle collisions the hash table can be restructured where each hash location can accommodate more than one item that is each location is a “bucket” or an array itself. Another method is to design the hash table as an array of linked chains.However there are other methods where the items are stored at alternative

locations in the hash table itself. Based on the above discussions there are basically two broad categories of collision resolution based on where the item that caused the collision is stored. One category of collision resolution strategy called **open hashing or separate chaining** stores the collisions outside the table. In the case of **closed hashing or open addressing** another slot in the table is used to store the keys that result in collisions. According to the method by which another slot is determined when collision occurs we will be discussing three popular rehashing methods namely Linear Probing, Quadratic Probing and Double Hashing.

18.3 Collision Resolution Strategy – Chaining

In collision by chaining the hash table itself is restructured where a separate list of all elements that hash to the same value is maintained. In other words chaining sets up lists of items with the same index. The array elements are pointers to the first nodes of the lists. When a new item hashes to a slot in the table it is inserted to the front of the list.

In this method a linked list is built for each bucket, and a linear search is conducted within each list to find an element that hashes to the index of the array that points to that particular list (Figure 18.3). This method is simple and widely used.

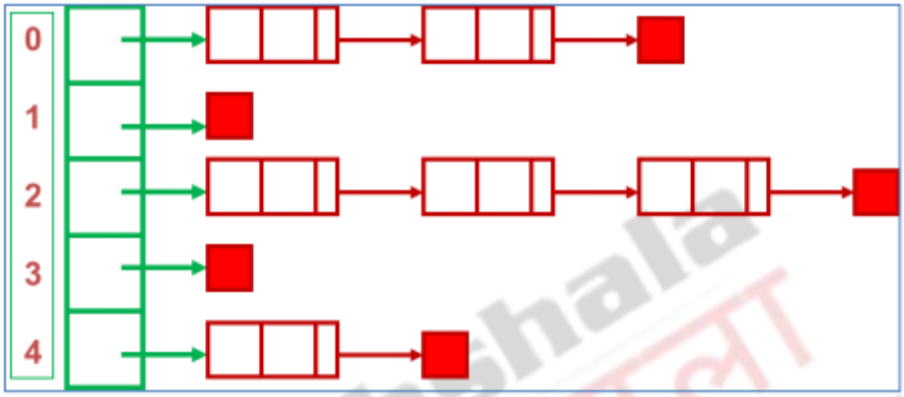

Figure 18.3 Collision Resolution with Chaining

18.3.1 Operations

The following are the operations associated with the hash table with chaining.

Initialization: all entries are initially set to NULL

Find: We first locate the cell in the array using hash function and then conduct a sequential search on the linked list pointed to by that cell. The ratio of the number of elements N stored in the hash table to the size of the Hash Table M is called the load factor L. M is the number of slots in the table which in turn is equal to the number of linked lists. The load factor L=N/M. The average length of a list will also be N/M. For successful search, the list contains the node that stores the searched item and 0 or more other nodes. On the average we need to check half the number of nodes in the list while searching for a certain element. Therefore the average search cost for a successful search is of O(N/2M). However for an unsuccessful search we need to traverse the entire list and hence the average search cost for an unsuccessful search is of O(N/M). This method cuts search time by a factor of M over sequential search in case of successful search but however requires requires extra memory outside of table.

Insertion: We first locate the cell in the array using hash function and if the item does not exist we insert it as the first item in the list. The time for insertion is O(1) since we hash to the array element and always insert at the front of the linked list. If the lists are kept sorted then an unsuccessful search is also of O(N/M) only, however in this case the time for insertion is of O(N).

Deletion: We first locate the cell in the array using hash function and then search for the item in the linked list and then delete the item from the linked list.

18.3.2 Advantages

Collision handling is simple as it only involves searching a linked list. The addition of lists to each slot of the hash table to store more items than the size of the table ensures that overflow does not generally occur. Since we are using linked lists, nodes required for inserting elements can be increased dynamically. Deletion is quick and easy as we are dealing with only deletion from the linked list. When we are handling large items there is better space utilization.\

18.4 Collision Resolution Strategy – Open Addressing

Separate chaining has the disadvantage of using linked lists and requires the implementation of a second data structure. The other category of collision resolution strategy is the open addressing hashing system(also called closed hashing) where all items are stored in the hash table itself. In this case in addition to the cell data (if any), each cell is also associated with one of the three states: EMPTY, OCCUPIED, DELETED. In this case we choose a bigger table whose size is results in a load factor below 0.5. The collision resolution strategy needs to generate a sequence of hash table slots testing each slot until an empty one is found. This process is called probing.

During insertion if a collision occurs, alternative cells are tried until an empty cell is found. More formally the slots $h0(x)$, $h1(x)$, $h2(x)$, ...are tried in succession where $hi(x) = (hash(x) + f(i))\text{mod}M$, with $f(0) = 0$ *where x is the key to be stored and M is the table size*. The function f decides the collision resolution strategy. Deletion is called lazy deletion because when a key is deleted the slot is marked as DELETED rather than removing the element physically. For searching a key, the same sequence of probing is used. If we reach a NULL pointer then it indicates that the element is not in table.

In open addressing, the colliding item is placed in a different available slot of the table obtained through probing. That is when we need to insert and we find the slot full, we need to try another slot until an open slot is found. This procedure is called probing. To conclude the probe sequence is the sequence of hash slots that are tried for finding an empty slot during insertion, or for finding the key during find or delete operations. We will discuss three common types of probing strategies namely Linear probing, Quadratic Probing and Double Hashing. We will discuss these probing strategies in subsequent sections.\

18.4.1 Advantages and Disadvantages of Open addressing

In Open Addressing all items are stored in the hash table itself. There is no need for another data structure. This method requires less storage than other types of hashing.

However Open Addressing also has several disadvantages. In this method it is better that the keys of the items to be hashed be distinct, however this is not always the case. The choice of a proper table size is an important factor that decides the

effectiveness of the method. It also requires the use of a three-state (Occupied, Empty, or Deleted) flag in each cell.

18.4 Rehashing Strategies associated with Open Addressing

Associated with open addressing is a rehash strategy that is in order to insert, if slot where the insertion is to be carried out is full, try another slot and continue to do so until an open slot is found. This probing or rehashing can be defined as follows: "If one tries to place x in bucket $h(x)$ and find it occupied, find alternative location $h1(x)$, $h2(x)$, etc. Try each in order, if none of the locations are empty then it indicates that the table is full." In this context, $h(x)$ is called *home bucket*. There are many rehashing strategies. We will be discussing three strategies namely **Linear Probing, Quadratic Probing and Double Hashing.**

18.4.1 Linear Hashing (Linear Probing)

The simplest rehash strategy is *linear hashing*. Here collisions are handled by finding the next available table slot for the colliding item. (Figure 18.4). So to find location of key k:

$$hi(k)= (h(k) + i) \text{ mod } M \text{ where } i=0,1,2.....$$

Here M is the size of the table. Here we compute hash value and increment it until a free cell is found. Each table cell inspected is referred to as a "probe". Generally uses less memory than chaining since we do not have to store all the links. However this method is slower than chaining since we may have to walk along table for a long way in order to find the item. Deletion requires the marking of the slot as deleted or filling the slot by creating space by shifting elements. The second method is not used since it is time consuming.

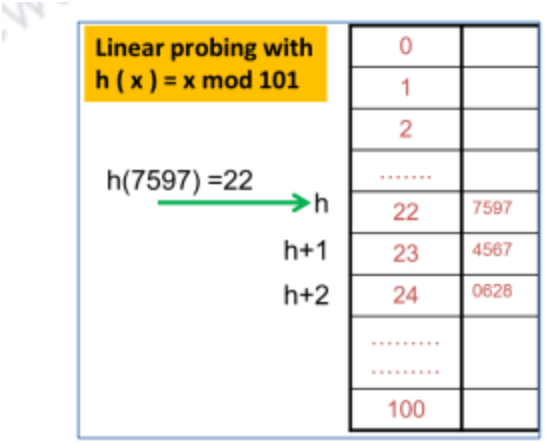

Figure 18.4 Linear Hashing or Probing

18.4.1.1 Search with Linear Hashing

In the case of linear probing, we need to search the hash table until the key or an empty slot is found which indicates that the key is not present. In the case of searching for insertion consider a hash table **A** that uses linear hashing. To find the key k using find(***k***), we start at slot ***h*(*k*)**. If the key k is found then no rehashing occurs. If collision occurs, we probe consecutive locations until one of the following occurs: An item with key ***k*** is found, or an empty slot is found, or ***all*** slots have been probed without success.

18.4.1.2 Updates with Linear Probing

To handle insertions and deletions, the location is tagged with the special flag, called ***DELETED***. The steps for removing an element are given in Figure 18.5. Here we assume that item **o** associated with key k is deleted.

---

removeElement(*k*)

- We search for an item with key ***k***
- If such an item (***k***, ***o***) is found, we replace it with the special item ***DELETED*** and return the position of the item
- Else, we return a null position

Figure 18.5 Removing an Element

To handle collisions when performing insertion we use linear probing to find the next available location. The steps for inserting an element are given in Figure 18.6. Here M is the size of the size of the table.

---

linear_probing_insert(K)

```
if (table is full) error
probe = h(K)
while (table[probe] occupied)
probe = (probe + 1) mod M
table[probe] = K
```

Figure 18.6 Inserting an Element

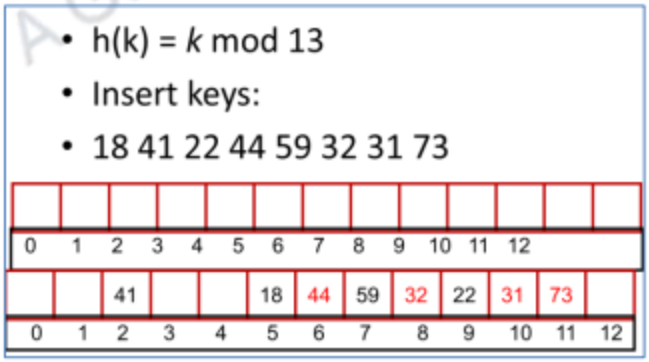

Figure 18.7 Example of Insertion using Linear Hashing

In the example (Figure18.7) the hash function used is mod 13. In this example we assume that the of the table is [0,1,2...12]. Now for the first insertion of 18, the key hashes to 5 and is inserted since there is no collision. Next 41 is inserted similarly at location 2. Next 22 hashes to location 9. Now when we try to insert 44, this hashes to location 5 and collision occurs. Hence we carry out linear probing by searching for next location which is empty and so 44 is inserted at location 6. Next 59 hashes to location 7, since there is no collision, it is inserted there. Insertions of 32, and 73 result in collisions and locations are found using linear hashing.

Runtime for the insertion algorithm using linear probing will be of theO(1) only if table is sparse. However as table fills, clustering occurs that is long clusters tend to get longer. Therefore insertion becomes very slow when the table is 70-80% full. In other words colliding items lump together, causing future collisions to cause a longer sequence of probes.

18.4.2 Quadratic Probing

The next type of rehashing or probing we will consider is Quadratic Probing. Here of the primary clustering problem associated with linear probing is avoided. The popular choice is $f(i) = i2$ that is where we increment by $i2$ instead of i .

If the hash function evaluates to h and collision occurs at slot h, the slots h + 12, h+22, ... h + i2 are probed. In other words in this type of probing the cells 1,4,9 and so on away from the original probe location are examined. Figure 18.8 shows an example of quadratic probing where the original key is placed at location 22 and the next probed locations are 22 + 12(23), 22+22(26), and 22+ 32(31).

18.4.2.1 Issue associated with Quadratic Probing:

It is possible that we may not probe all locations in the table and there is no guarantee that an empty cell will be found even if the table is a little more than half full. In case the size of the hash table is not a prime number this problem is even more severe. However, if we choose the table size to be a prime number and the load factor (discussed earlier) is larger than 0.5 all probes will be to different locations and it is possible to find a location to insert the element. With load factor near 0.5, the expected number of probes is near optimal, though this cannot be proved. Though there will be no clustering with similar keys, clustering is still possible with identical keys.

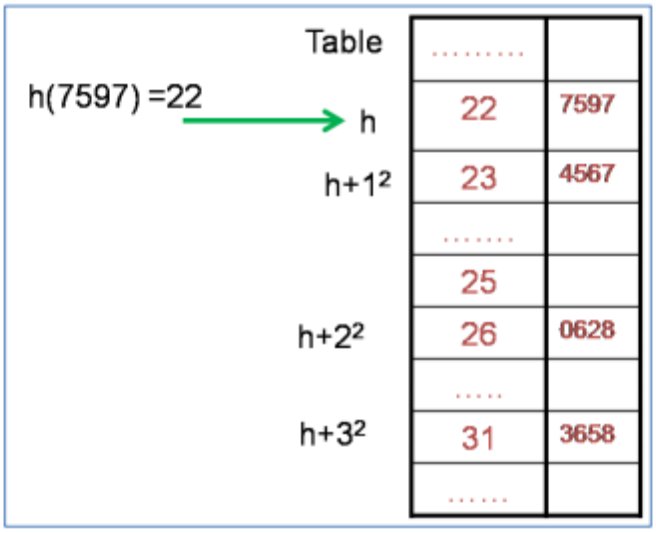

Figure 18.8 Quadratic Probing with h (x) = x mod 101

18.4.3 Double Hashing

The last method of probing we will discuss is the method of Double hashing that is we use two hash functions. As the name indicates we will use a secondary hash function ***h2(k)*** and handle collisions by placing an item in the first available cell of the series:

$h(k,i) = (h1(k) + ih2(k)) \bmod M$ for $i = 0, 1, \ldots, M-1$.

The secondary hash function *h2(k)* is an offset that cannot have zero values. The table size **M** must be a prime number to allow probing of all the cells. The primary has function h1(k) =kmodM. The secondary hash function can be h2(k) = q where q is taken to be generally less than M, and is prime. This method distributes keys more uniformly than linear probing. The steps of insertion into a hash table using double hashing are given in Figure 18.8. Figure 18.9 shows the insertion of the keys 58, 14 and 91 using double hashing. The insertion of 58 causes no collision and 58 is inserted into location 3. Now 14 also hashes to the same location 3 using h1(k) and a collision occurs. Now we probe with (h1(14) +7))mod 11 that is (3+7) mod 11 which gives us location 10, which is empty so 14 is inserted there. Now we want to insert.We find the locations h1(91) and (h1(91)+7)mod 11 are both occupied so we probe further with (h1(91)+2*7)mod11 that is (3+2*7) mod 11 that is location 6 which is empty and so 91 is placed there.

```
double_hash_insert(K)
if(table is full) error
probe = h1(K)
offset = h2(K)
while (table[probe] occupied)
    probe = (probe + offset) mod M
table[probe] = K
```

Figure 18.8 Inserting an element with Double Hashing

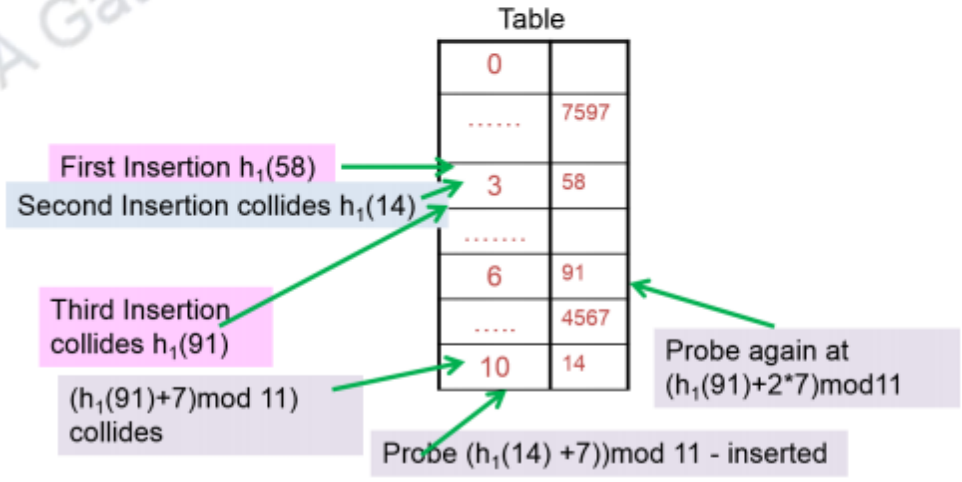

Figure 18.9 Double hashing during the insertion of 58, 14, and 91

Summary

- Explained the concept of collision in hashing
- Discussed the different collision resolution methods such as separate chaining and open addressing methods.
- Discussed the three probing methods of open addressing such as linear probing, quadratic probing and double hashing with respect to time and space requirements.

---

you can view video on Hashing and Collisions

Web Links
ocw.metu.edu.tr/file.php/90/**Hashing.ppt**
https://courses.cs.washington.edu/courses/cse326/02au/.../part5-**hashing**.ppt
www.cse.unt.edu/~huangyan/3110/Lectures/**Hashing.ppt**
faculty.kfupm.edu.sa/ICS/jauhar/ics202/Unit29_Hashing2.**ppt**
research.cs.queensu.ca/home/cisc235/DW-Topic5HashTables.**ppt**
faculty.washington.edu/moishe/tcss342/HashingFinal.**ppt**
http://www.cs.umsl.edu/~sanjiv/classes/c
http://www.mif.vu.lt/~algis/dsax/DsHash.
http://www.cs.purdue.edu/cgvlab/courses/
Supporting & Reference Materials
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Adam Drozdek,"Data Structures and Algorithms in C++", Cengage; 4th edition,2013
Alfred V. Aho and Jeffrey D. Ullman, "Data Structures and Algorithms", 1983
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014